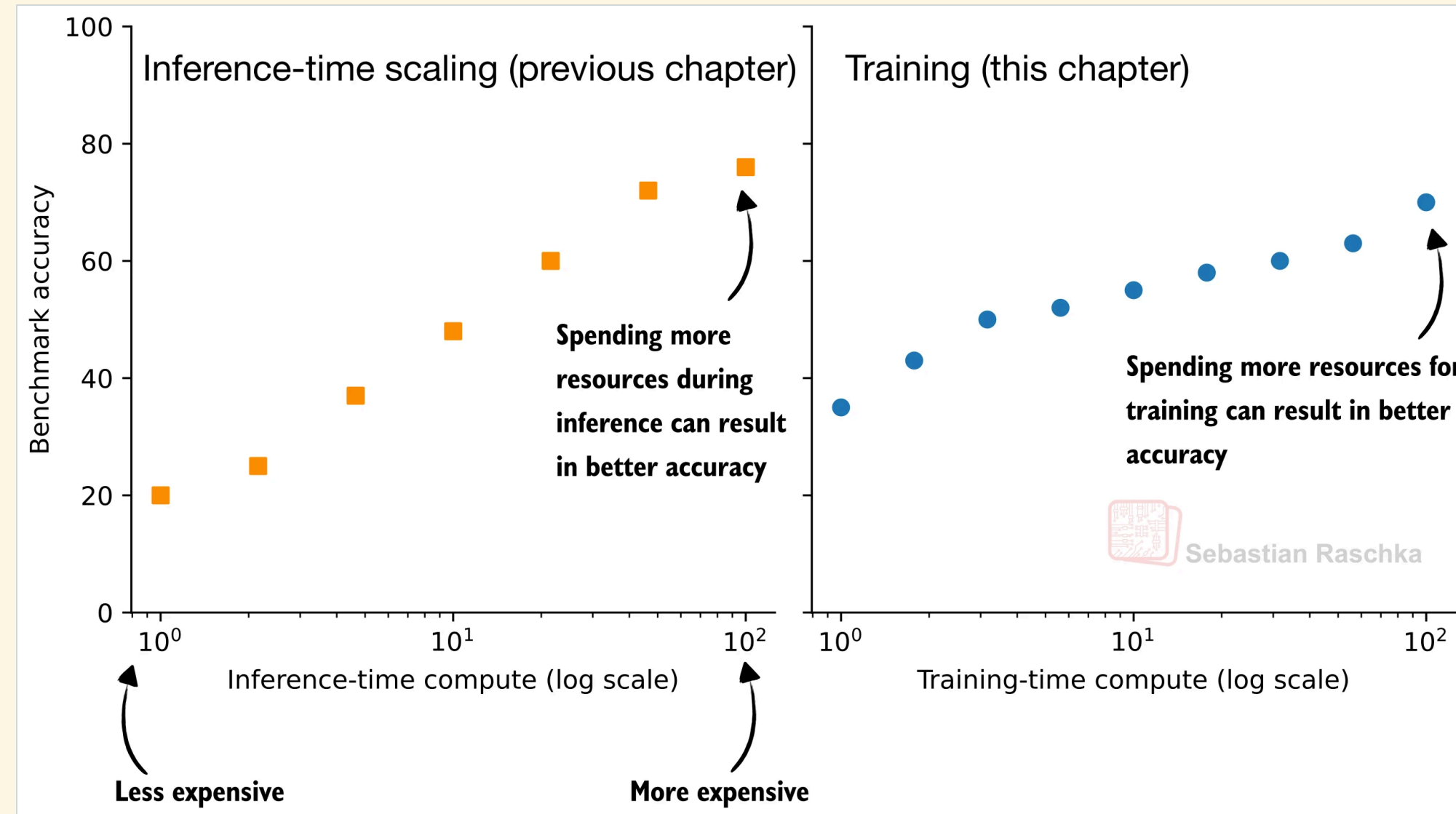


Reinforcement Learning for LLMs

RL · RLHF · RLVR · GRPO

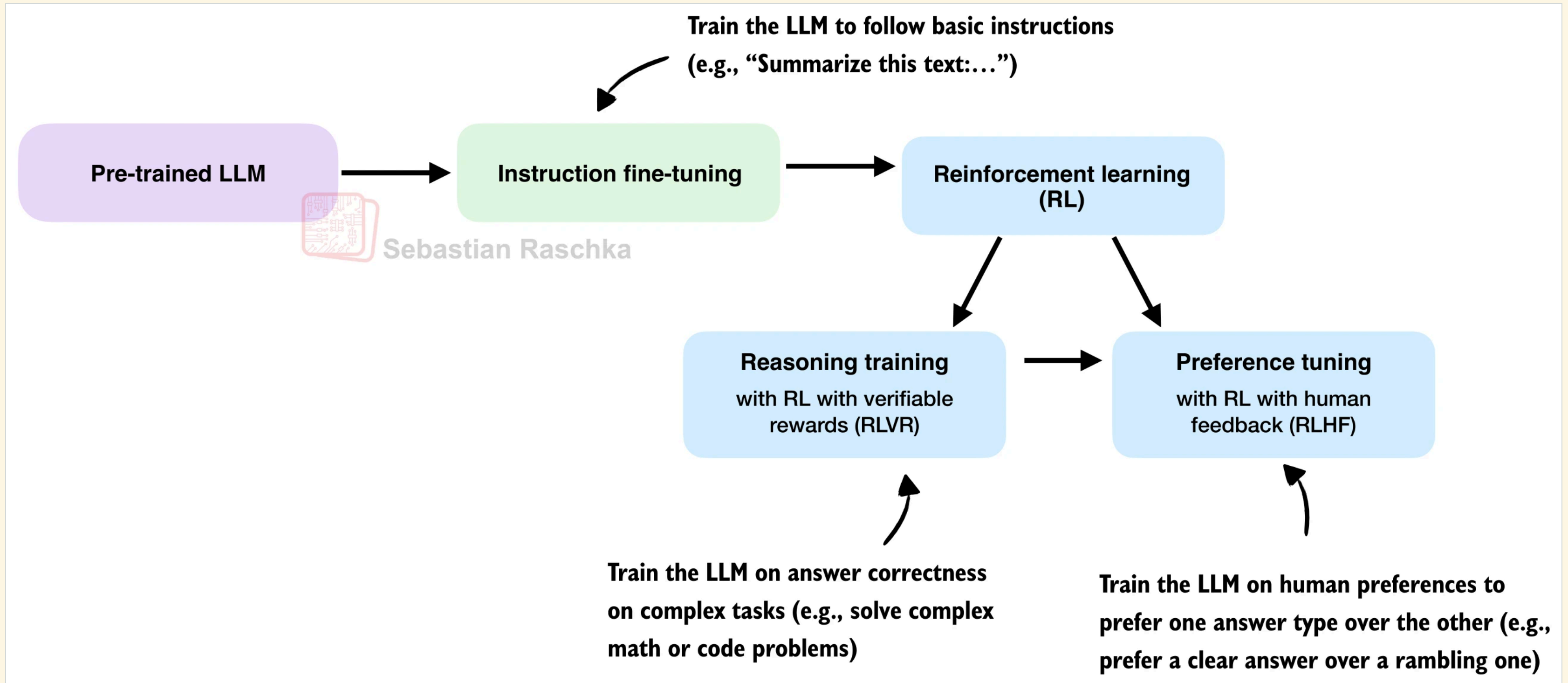
Dhruv Kumar · BITS Pilani

Two ways to spend compute for better reasoning



Inference-time scaling spends more compute per generated answer (sampling, voting, self-refinement). **Training-time scaling** spends more compute during training itself — reinforcement learning is exactly this, and is this deck's focus. In practice, the two are combined: RL-trained models still benefit from inference-time techniques applied on top.

Where RL fits in the LLM training pipeline



RL is applied as a **post-training** stage, typically after instruction fine-tuning. It then splits into two distinct uses: **RLVR** trains on answer correctness for complex tasks like math or code; **RLHF** trains on human preferences between response styles. Reasoning-focused RL can also be applied directly to a base model – weaker overall, but a simpler setting for isolating what RL alone contributes (this is how DeepSeek-R1-Zero was trained).

Pretraining vs. RL: different objectives

Pretraining / SFT

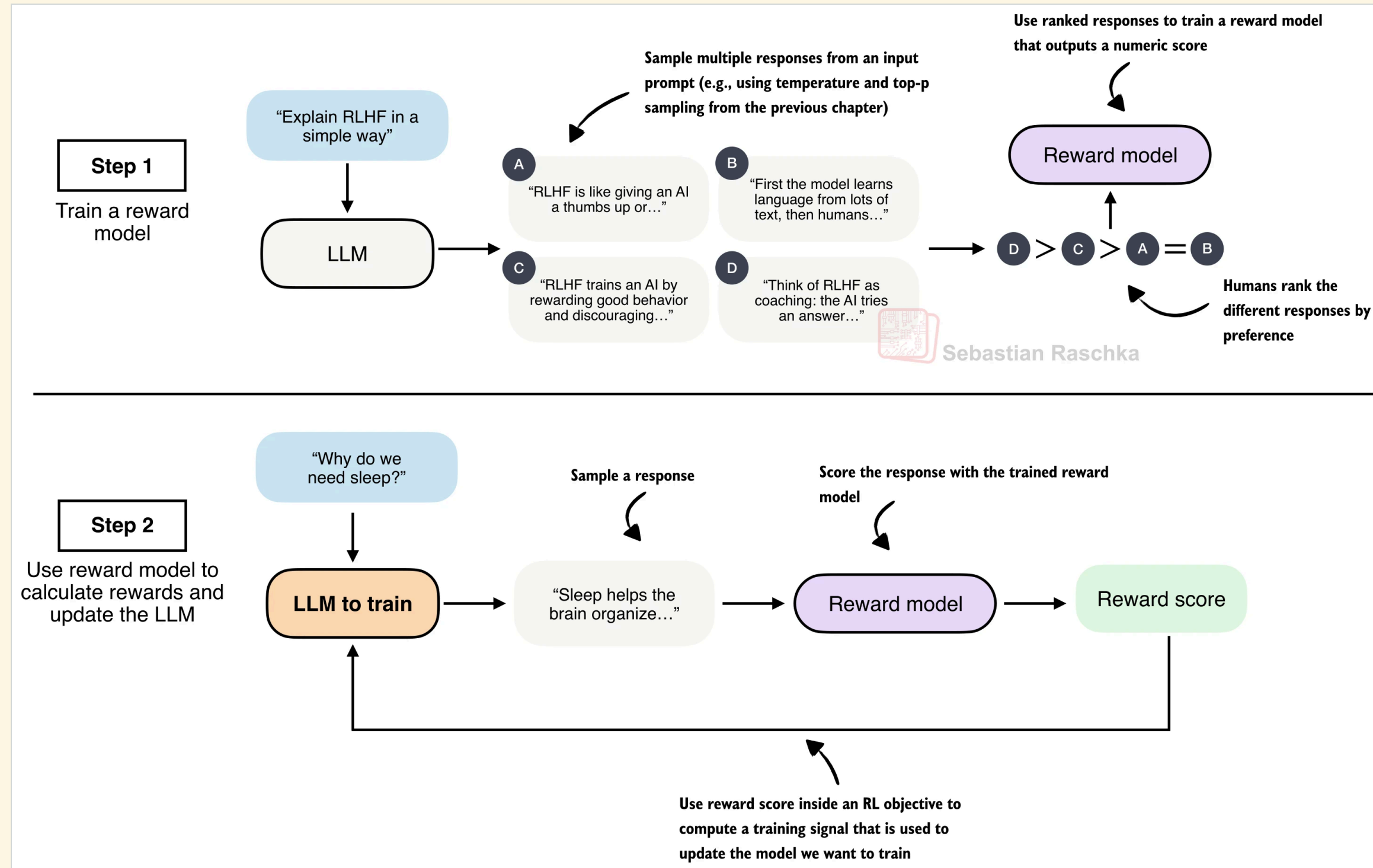
Optimizes **next-token prediction** — build general knowledge and language ability by predicting the next token given everything before it.

Reinforcement learning

Optimizes **sequence-level objectives** — refine model behavior using a signal defined over the whole generated response, such as answer correctness or human preference, not any single next token.

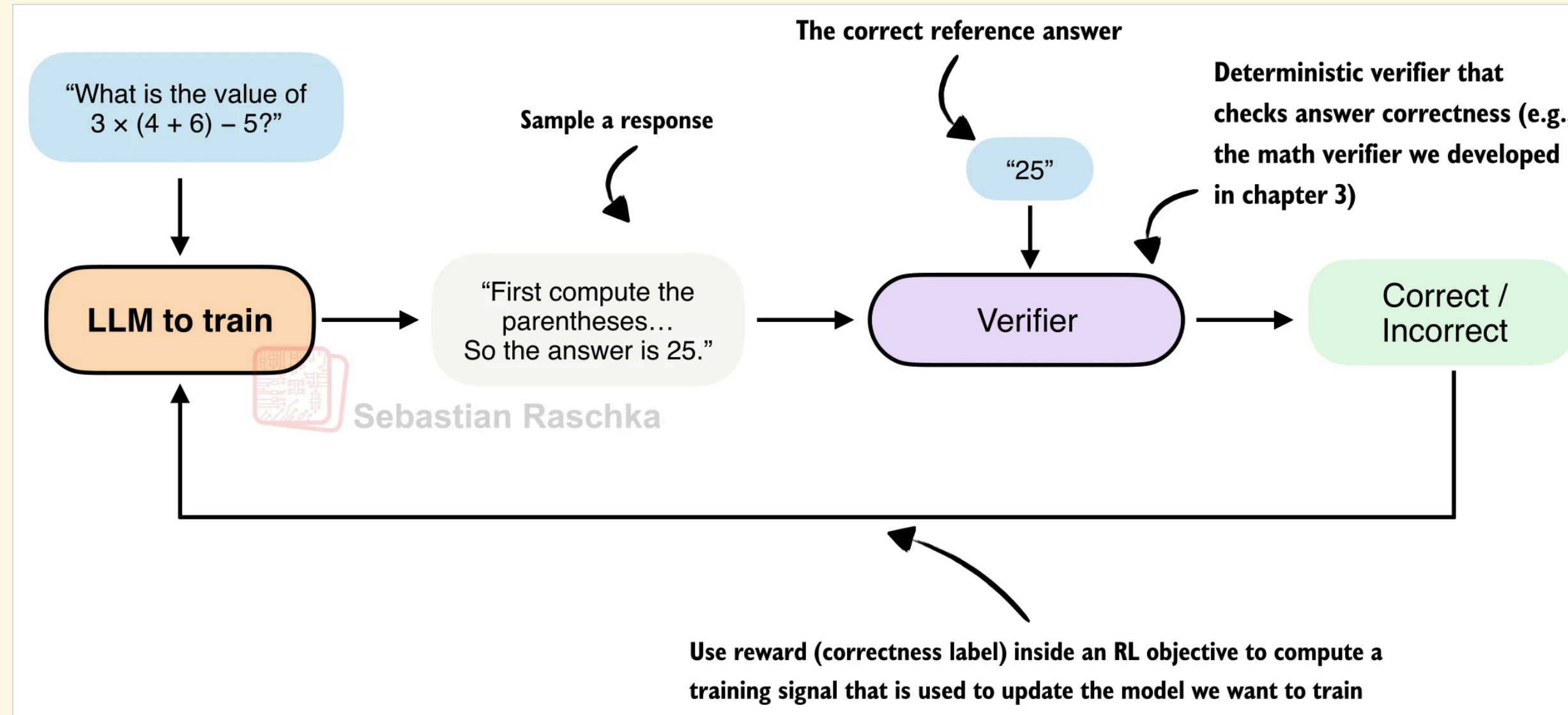
This shift — from scoring individual tokens to scoring entire responses — is what the rest of this deck unpacks concretely, through GRPO.

RLHF: the original pipeline



Introduced in the 2022 InstructGPT work — the key step that turned GPT-3 into the original ChatGPT. Unlike pretraining and SFT, which optimize next-token prediction, RLHF optimizes the model based on **human preference labels** over its own responses — typically via a separately trained reward model that scores full responses, which the policy is then updated against.

From human feedback to verifiable rewards: RLVR



RLHF's reward model is itself a large, expensive-to-train LLM. **RLVR** replaces it with an automatically verifiable, deterministic reward — e.g. checking whether a math answer is correct, or whether code compiles and passes tests. DeepSeek-R1 (2025) popularized this approach, achieving strong reasoning without any human preference data or learned reward model. This deck focuses on math-based verification, but the same idea extends directly to code verification.

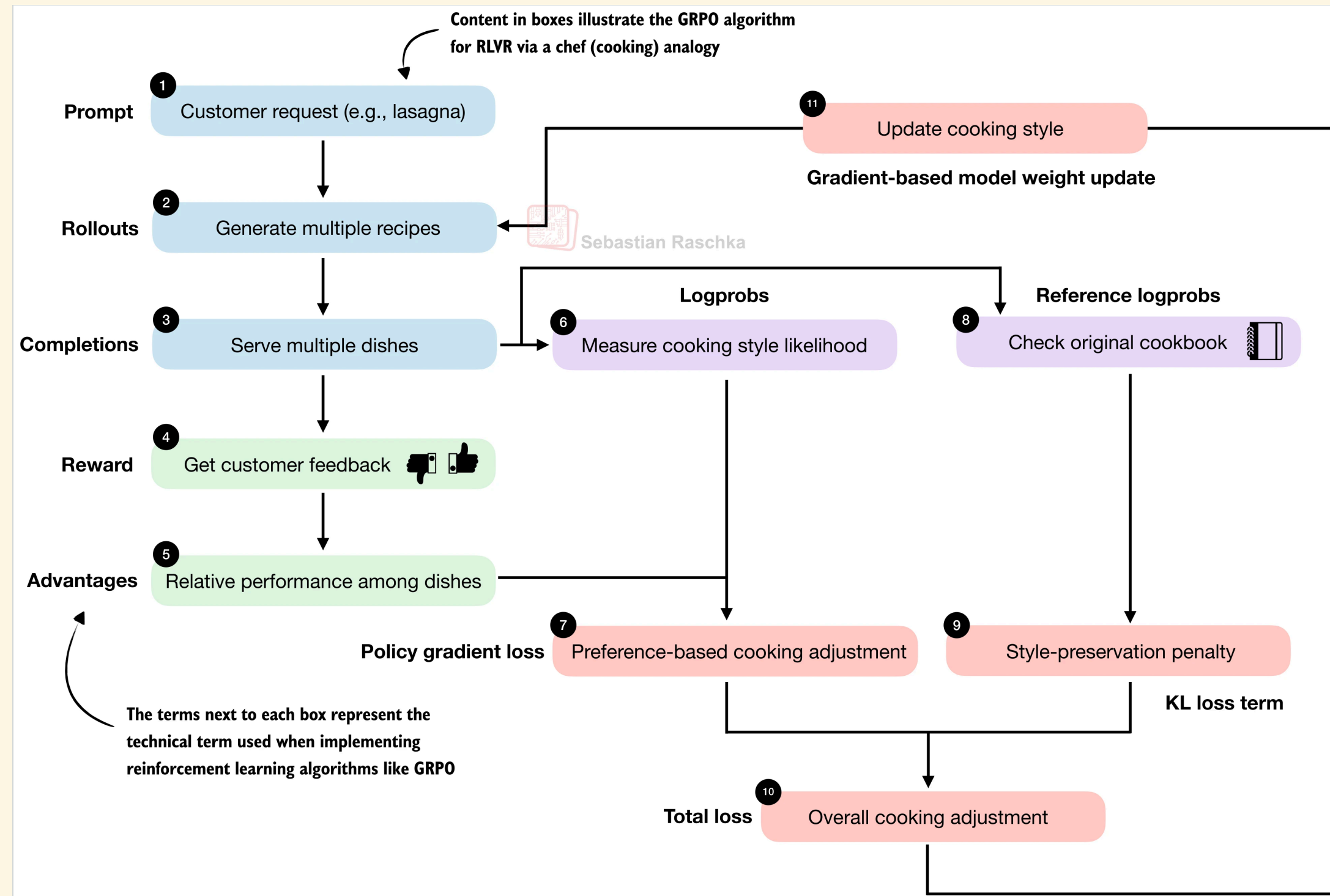
The policy gradient algorithm: why GRPO, not PPO

RL for LLMs updates the model — called the **policy** — using a policy gradient algorithm. Proximal Policy Optimization (PPO) is the standard choice for RLHF and works for RLVR too. DeepSeek instead used a simpler algorithm for training DeepSeek-R1: Group Relative Policy Optimization (**GRPO**), first introduced in DeepSeekMath.

WHY GRPO IS MORE RESOURCE-FRIENDLY

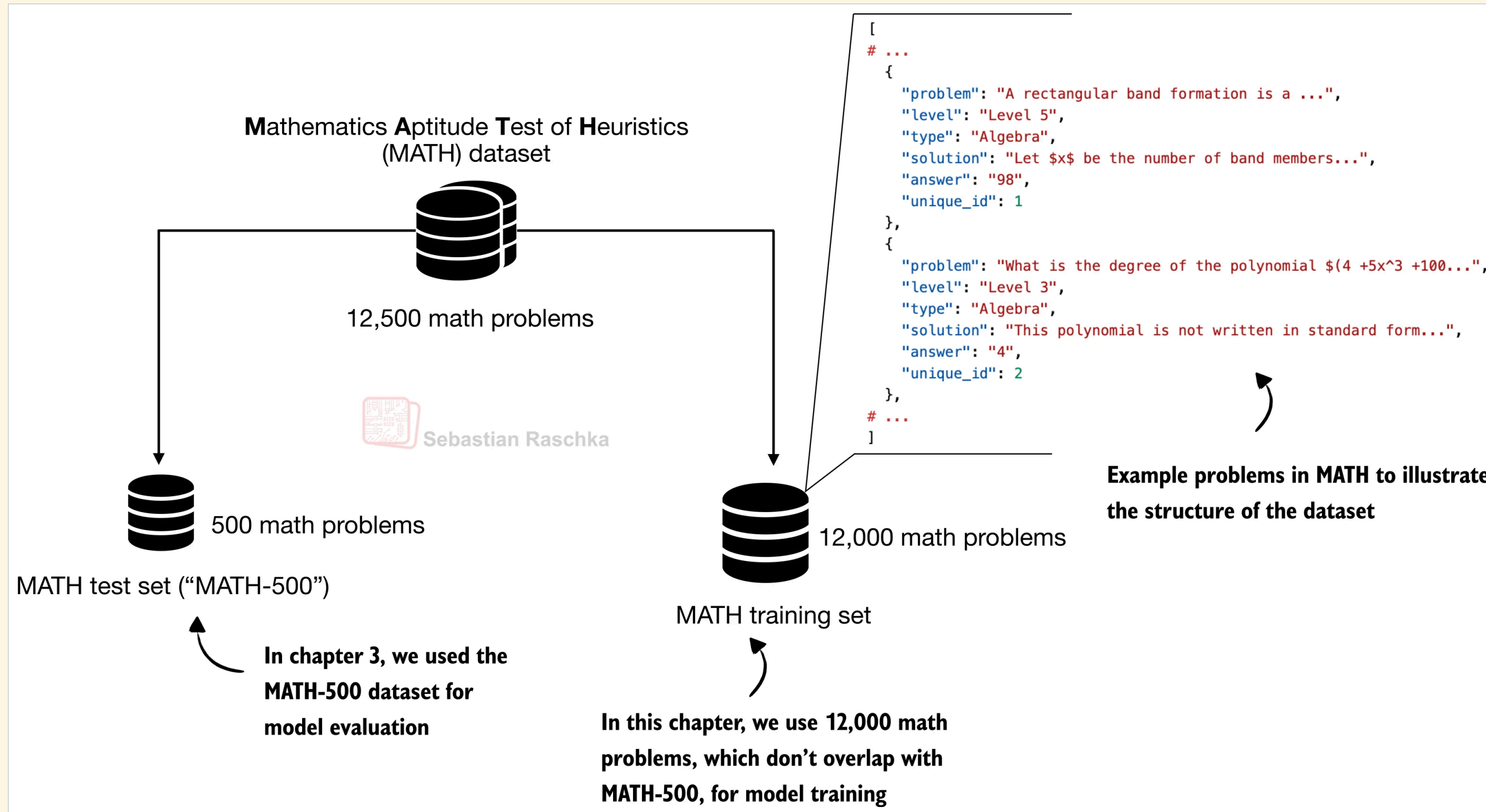
PPO trains a separate LLM to estimate a value function. GRPO needs no such model — it derives its learning signal purely from **relative comparisons within a group of sampled responses** to the same prompt. One fewer large model to train and hold in memory during RL.

GRPO intuition: the chef analogy



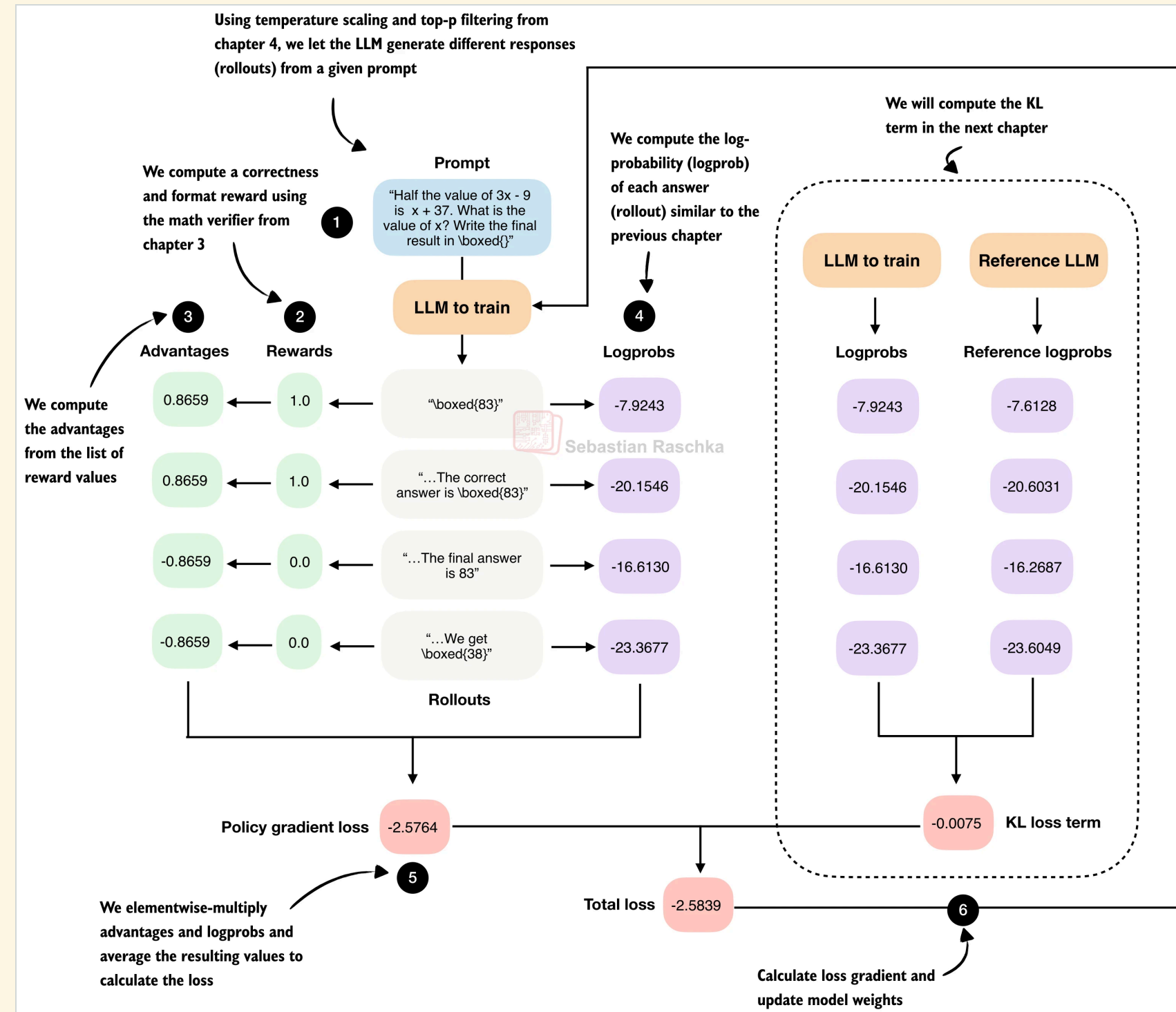
A chef (the **policy**) cooks several variations of a dish (**rollouts**, also called completions) from the same recipe (**prompt**), tastes and scores each (**rewards**), and adjusts technique based on which dishes did better or worse than the average of the batch (**advantages**) — not against some fixed external standard. This relative, within-group comparison is the "group relative" in GRPO.

Training data: a MATH subset



A non-overlapping training split of the MATH dataset, explicitly excluding the MATH-500 examples reserved for evaluation. Only the `problem` and `answer` fields are used — the `solution` field is deliberately withheld, so the model must explore its own solution paths rather than imitate a fixed one.

GRPO end to end: the full computation



One real worked example, four rollouts for the prompt "Half the value of $3x - 9$ is $x + 37$. What is x ?" The next several slides walk through exactly how each number here is produced, stage by stage.

Stage 1–2: rewards from a verifier

Each rollout is scored by an automatic, deterministic verifier — the core idea of RLVR. Here the reward is **correctness**, with an implicit **format** requirement: a reward of 1.0 is given only if the final answer appears inside `\boxed{}` and is correct; otherwise the reward is 0.0.

ROLLOUT	REWARD
"\boxed{83}"	1.0
"...The correct answer is \boxed{83}"	1.0
"...The final answer is 83" (<i>not boxed</i>)	0.0
"...We get \boxed{38}" (<i>wrong answer</i>)	0.0

DeepSeek-R1's authors also tried scoring intermediate reasoning steps with a process reward model — this was unsuccessful, and they found it better to train on final-answer correctness alone, with no intermediate rewards.

Stage 3: turning rewards into advantages

The "group relative" step: each rollout's reward is normalized against the mean and spread of rewards *within its own group* (the rollouts sampled for the same prompt):

$$\text{advantage}_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon}$$

r_i — reward of the i -th rollout; μ_r — mean reward across the group; σ_r — standard deviation of rewards across the group; ϵ — a small constant preventing division by zero.

INTUITION: GRADING ON A CURVE

Some prompts are easy (most rollouts already correct) and some are hard (most wrong) — a raw reward of 1.0 alone doesn't say whether a rollout was impressive. Subtracting the group mean asks a sharper question: *did this rollout beat its siblings on the very same problem?* Dividing by the group's spread then puts every prompt's signal on a common scale, so one unusually spread-out group can't dominate training just because its raw rewards happened to vary more.

A BUILT-IN STOPPING SIGNAL

If every rollout in a group gets the same reward — all correct, or all wrong — then $r_i - \mu_r = 0$ for every i . The model receives **no update** in that step. This is a feature: once every sampled answer is already correct, there is nothing left to learn from that prompt.

Verified: rewards → advantages

ROLLOUT	REWARD R_I	ADVANTAGE
"\boxed{83}"	1.0	0.8659
"...The correct answer is \boxed{83}"	1.0	0.8659
"...The final answer is 83"	0.0	-0.8659
"...We get \boxed{38}"	0.0	-0.8659

$\mu_r = 0.5$; the **sample** standard deviation ($\div (N-1)$, not $\div N$) gives $\sigma_r = 0.5774$, so $(1.0 - 0.5)/0.5774 = 0.8659$ – confirmed by direct computation, matching the figure exactly. Note some GRPO variants (e.g. Dr. GRPO) argue against dividing by σ_r at all, since it can overweight low-variance groups.

Stage 4: scoring rollouts with log-probabilities

A familiar quantity, the average per-token log-probability of the answer:

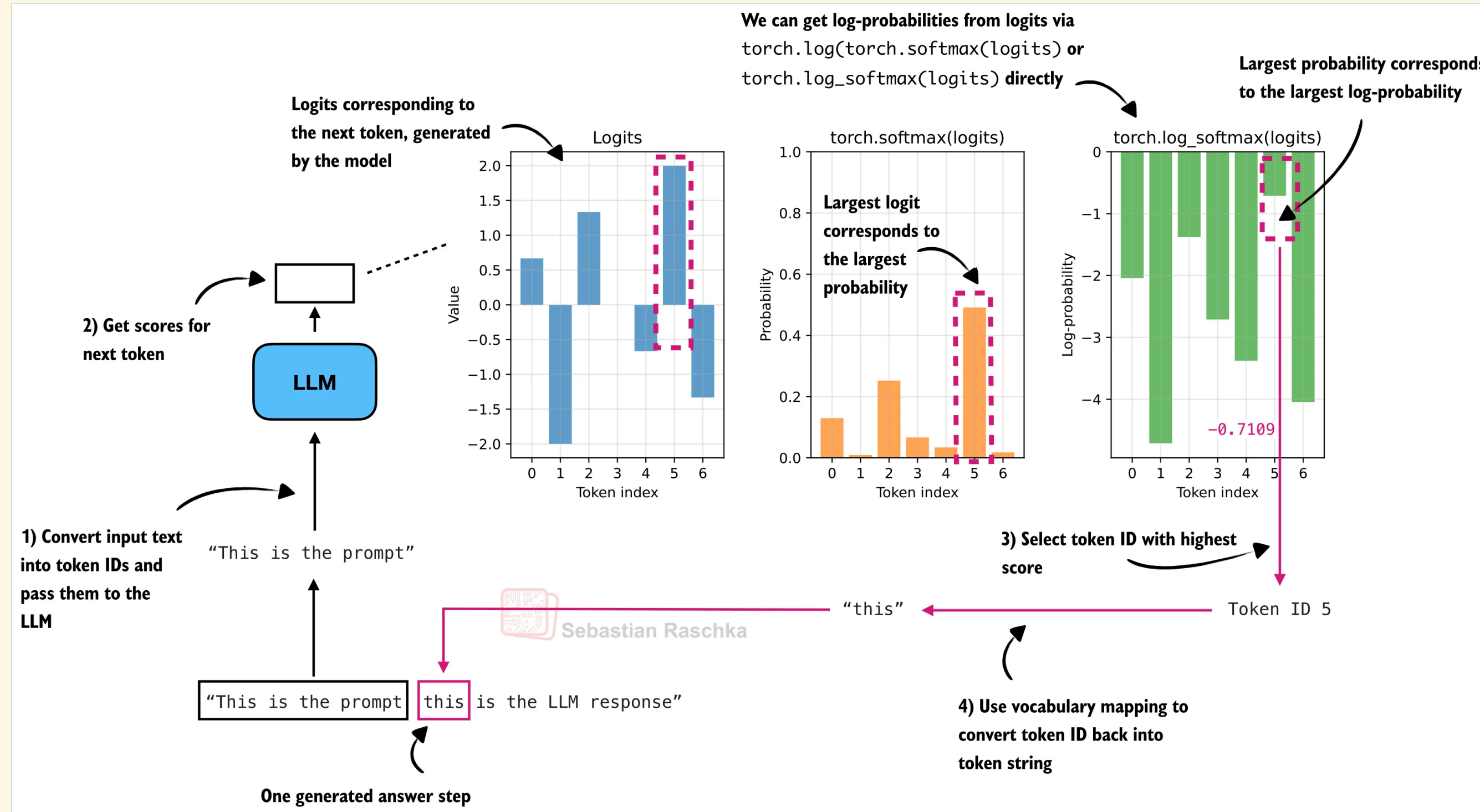
$$\frac{1}{T} \sum_{t=1}^T \log p_W(y_t \mid y_{<t}, x)$$

$y_1, \dots, y_T$ — tokens of the generated response, length T ; $y_{<t}$ — tokens generated so far; x — the input prompt; W — the model's weight parameters. Averaging over T length-normalizes the score, which is standard practice for comparing answers of different length.

GRPO DOES NOT USE THIS AVERAGED FORM

GRPO assigns one reward and one advantage to an entire rollout — to scale the gradient correctly, the log-probability must reflect the likelihood of the **full sequence**, obtained by *summing*, not averaging, the per-token log-probabilities. Averaging here would implicitly rescale the learning signal by response length and distort the update.

Where a per-token log-probability comes from



At each generation step, the model outputs one logit per vocabulary token. `torch.log_softmax(logits)` gives the log-probabilities directly (equivalent to, but more numerically stable than, `torch.log(torch.softmax(logits))`). The sequence-level log-probability from the previous slide is just the sum of these values, one per generated token.

Stage 5: the GRPO policy gradient loss

$$\mathcal{L}_{\text{PG}} = -\frac{1}{N} \sum_{i=1}^N A_i \sum_{t=1}^{T_i} \log p_W \left(y_t^{(i)} \mid y_{<t}^{(i)}, x^{(i)} \right)$$

N — number of rollouts in the batch; $y_1^{(i)}, \dots, y_{T_i}^{(i)}$ — tokens of the i -th rollout; $x^{(i)}$ — its prompt; p_W — the policy (student model), parameterized by weights W ; A_i — the advantage of the i -th rollout. The inner sum is exactly the sequence-level log-probability from two slides ago; the outer average combines advantages and log-probabilities across all rollouts in the batch.

TWO IMPLEMENTATION DETAILS

A_i is **detached** — it is a fixed target, and gradients flow only through the log-probabilities. The loss carries a **negative** sign because PyTorch optimizers minimize by default, while this objective should be maximized.

Intuition: why advantage $\times$ log-probability?

This product is not arbitrary — it comes from asking how to increase expected reward when the only thing we control is a probability distribution we can sample from, but not differentiate through directly (sampling a token is not a differentiable operation):

$$\nabla_W \mathbb{E}_{y \sim p_W} [A(y)] = \mathbb{E}_{y \sim p_W} [A(y) \nabla_W \log p_W(y)]$$

This identity (the "score function" or REINFORCE trick) turns a gradient of an expectation into an expectation of *reward times the gradient of a log-probability* — something we *can* compute. GRPO's loss is a Monte Carlo estimate of the right-hand side, using the N sampled rollouts as samples from p_W and their advantages A_i standing in for $A(y)$.

READING THE SIGN: REINFORCE, OR SUPPRESS

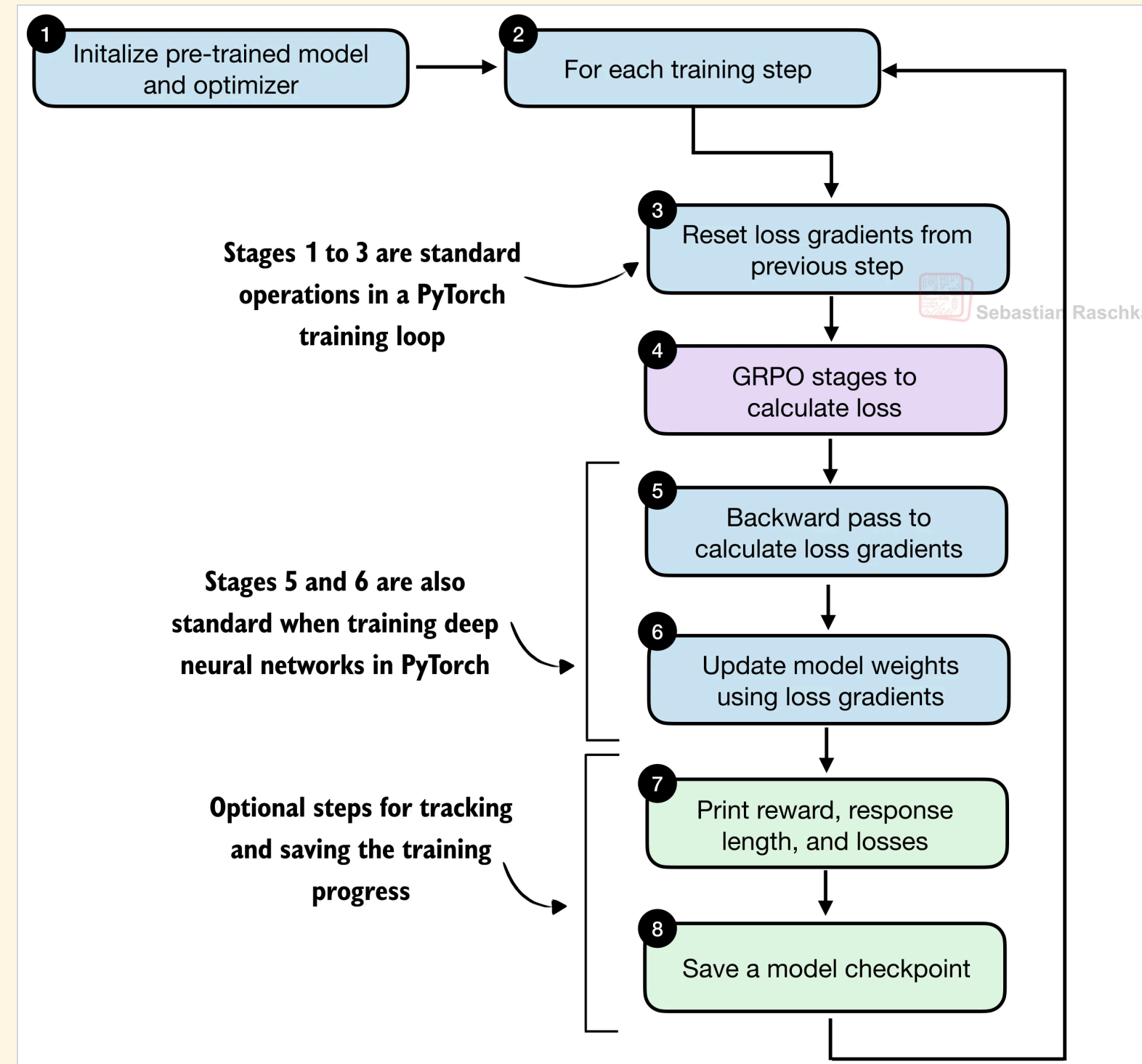
Think of $\log p_W(y_i)$ as a dial gradient descent can turn for each rollout. If $A_i > 0$ (better than the group average), the update turns the dial **up** — that exact response becomes more likely next time. If $A_i < 0$, the dial turns **down** — less likely. The size of $|A_i|$ sets how hard the dial is turned. Concretely, in this batch: "\boxed{83}" ($A = +0.8659$) gets pushed toward a less negative logprob than its current -7.9243 ; "\boxed{38}" ($A = -0.8659$, the wrong answer) gets pushed toward a more negative logprob than its current -23.3677 . This is literally the "reinforcement" in reinforcement learning — strengthen what worked, weaken what didn't, with no target sequence to imitate, only a relative comparison.

Verified: the full loss for this batch

ROLLOUT	ADVANTAGE A_I	SEQUENCE LOGPROB
"\boxed{83}"	0.8659	-7.9243
"...correct answer is \boxed{83}"	0.8659	-20.1546
"...final answer is 83"	-0.8659	-16.6130
"...We get \boxed{38}"	-0.8659	-23.3677

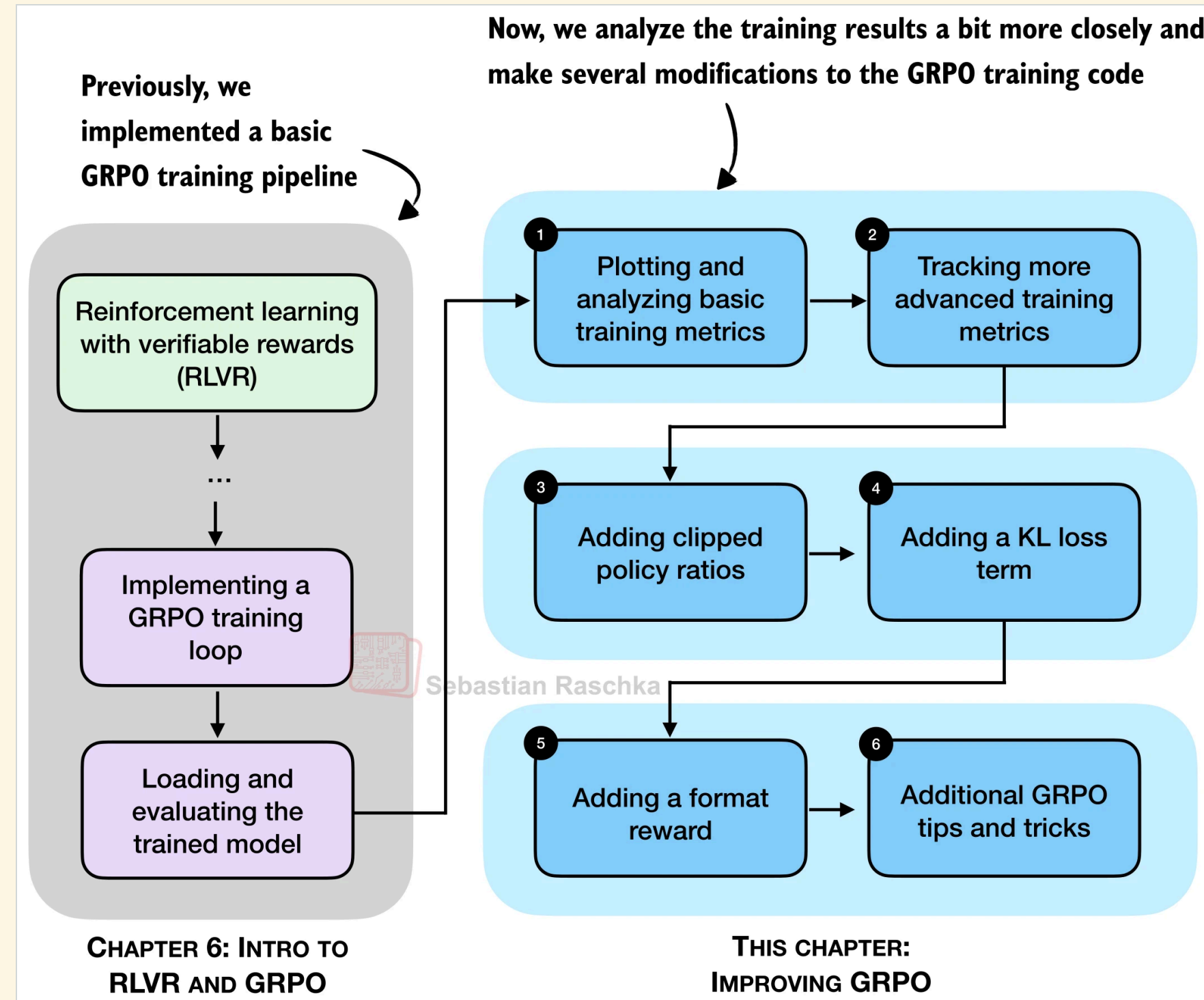
$\mathcal{L}_{PG} = -\frac{1}{4} \sum_i A_i \cdot \text{logprob}_i = -2.5764$ – confirmed by direct computation, matching the figure. Stage 6 backpropagates this loss and updates the model weights, exactly like any other PyTorch training step.

The GRPO training loop



Only the loss computation (purple) is specific to GRPO — resetting gradients, the backward pass, and the weight update are standard PyTorch training-loop operations, identical to training any other neural network.

Beyond the basics: stabilizing GRPO



The GRPO pipeline just covered is a working but simplified algorithm. The rest of this deck covers the improvements that make it more stable and effective in practice — better monitoring, clipped policy ratios, a KL regularization term, and an explicit format reward.

Tracking training health: the advantage standard deviation

By construction, the **mean** advantage across a group is always exactly 0 — tracking it is mostly a sanity check that the normalization code is correct. The **standard deviation** of the advantages is the genuinely informative signal:

std $\approx$ 1

A well-scaled gradient signal — healthy, stable updates.

std $\ll$ 1

A vanishing signal — often happens once rewards collapse to all-same values.

std $\gg$ 1

Overly spiky updates that can destabilize training.

Best read alongside the average reward: a reward average near 1.0 looks alarming (vanishing signal) but is actually good news — it means the model is already getting everything right.

Tracking training health: entropy

Entropy measures how uncertain the model is when producing the next token.

HIGH ENTROPY

Probability mass spread over many tokens — good for exploration, but too high means the model is close to random.

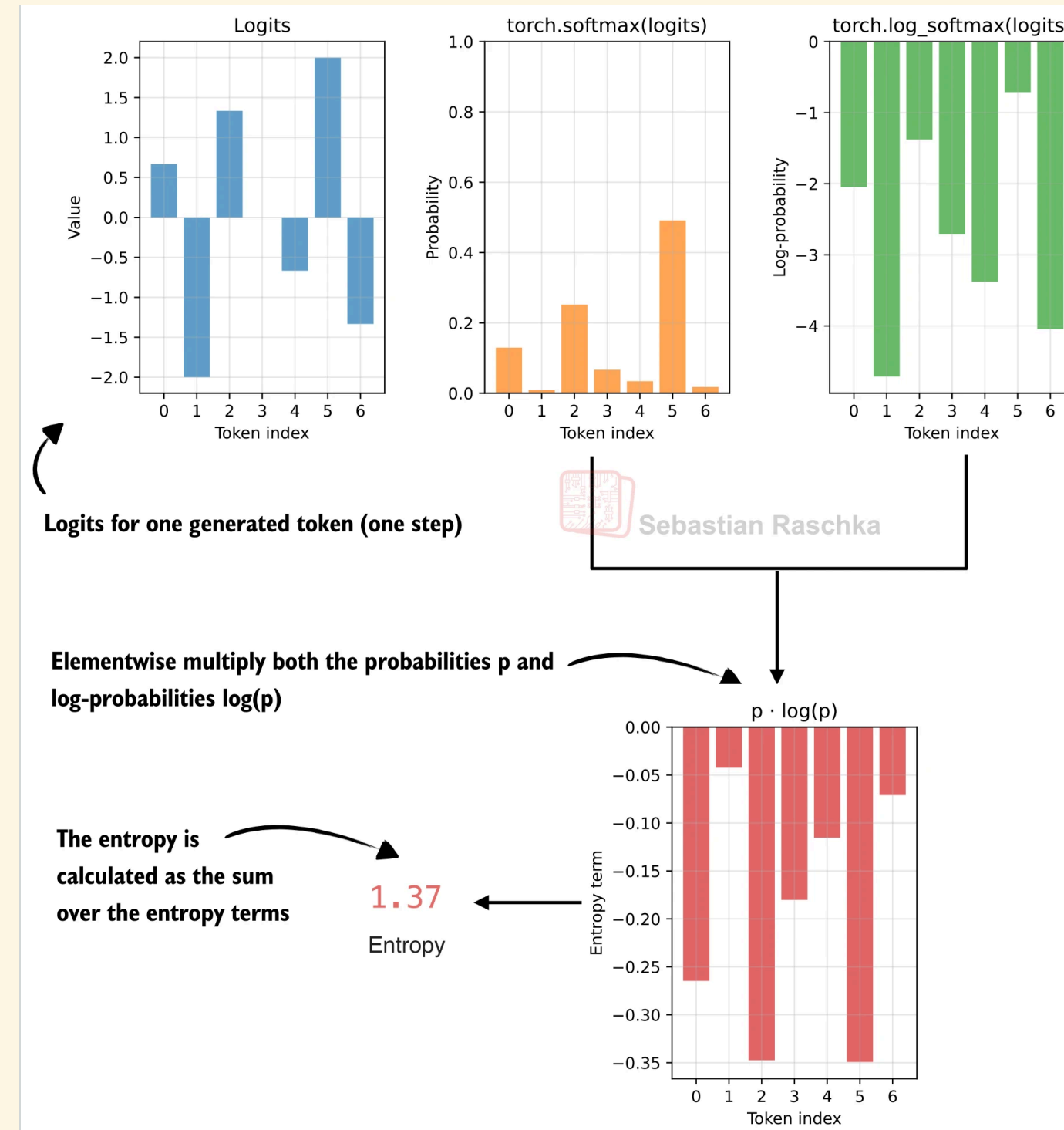
LOW ENTROPY

Almost all mass on one token — more deterministic, but very low entropy can signal collapse: the model repeating the same or near-identical outputs.

RULE OF THUMB (7-TOKEN VOCABULARY EXAMPLE)

≈0–0.5: one token dominates, near-deterministic. **≈1–2**: mass shared among a few tokens, typical during stable training. **» 2** (approaching $\log(7) = 1.9459$): highly uncertain, close to random.

Computing entropy from logits



$H = - \sum_v p_v \log p_v$ – probabilities and log-probabilities are multiplied elementwise, then summed (with a sign flip). In GRPO, the per-token entropy is averaged over every answer token in a rollout to get one entropy value per rollout, tracked over training just like the reward average.

Stabilizing updates: the clipped policy ratio

The clipped policy ratio measures how much the current policy differs from an earlier version of itself, by comparing sequence log-probabilities computed **before** an update step ("old") against those computed **after** it ("new") — borrowed directly from PPO (Schulman et al., 2017).

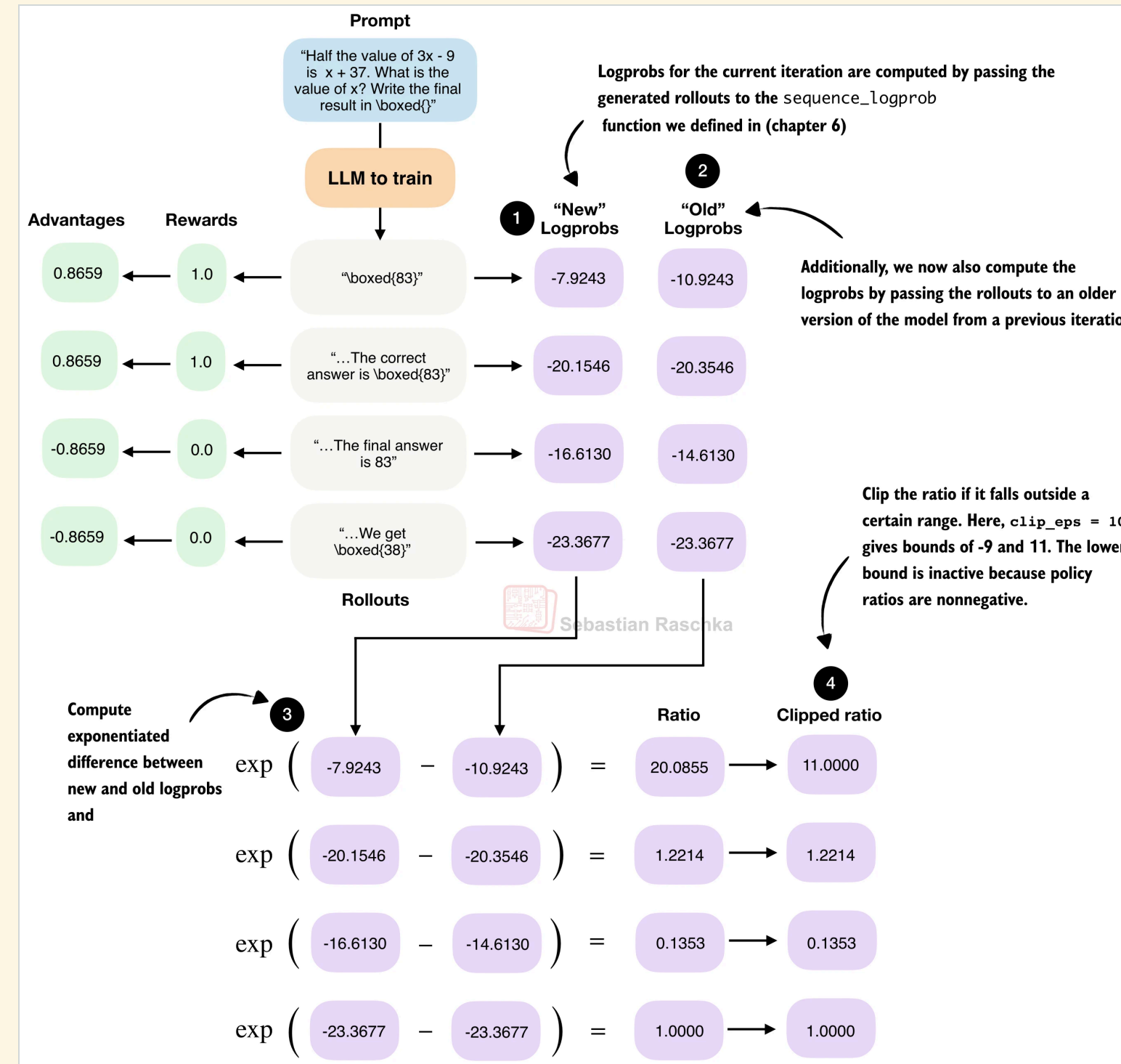
$$\text{ratio} = \exp(\text{logprob}_{\text{new}} - \text{logprob}_{\text{old}}), \quad \text{clipped ratio} = \text{clamp}(\text{ratio}, 1 - \epsilon, 1 + \epsilon)$$

The advantage is then scaled by this (clipped) ratio instead of being used directly, limiting how much any single rollout can move the policy in one update. DeepSeek-R1 used $\epsilon = 10$ (very weak clipping); other RL pipelines commonly use a much tighter $\epsilon = 0.1$.

INTUITION: DON'T BET EVERYTHING ON ONE LUCKY SAMPLE

A group has only a handful of rollouts — often just 4–8. By pure chance, one of them can end up looking dramatically more likely under the updated policy than it did under the old one. Left unchecked, that single noisy sample could dominate the entire update. Clipping acts like a seatbelt: a good rollout can still pull the policy toward it, but no single sample is allowed to pull it too hard in one step.

Verified: the clipped ratio in action



The first rollout's ratio of 20.09 gets clamped down to 11.0 — a case where the updated model became dramatically more confident in that rollout than the reference version was, exactly the kind of large, destabilizing jump clipping exists to contain.

Token-level vs. sequence-level ratios across papers

Different GRPO variants disagree on whether the policy ratio should be computed per token or per full sequence:

PAPER	LEVEL	NOTATION
DeepSeekMath GRPO (2024)	Token	$r_{i,t}(\theta) = \pi_{\theta}(o_{i,t} \mid \boldsymbol{q}, \boldsymbol{o}_{i,<t}) / \pi_{\theta_{\text{old}}}(o_{i,t} \mid \boldsymbol{q}, \boldsymbol{o}_{i,<t})$
DeepSeek-R1 GRPO (2025)	Sequence	$r_i(\theta) = \pi_{\theta}(o_i \mid \boldsymbol{q}) / \pi_{\theta_{\text{old}}}(o_i \mid \boldsymbol{q})$
DAPO (2025)	Token	$r_{i,t}(\theta) = \pi_{\theta}(o_{i,t} \mid \boldsymbol{q}, \boldsymbol{o}_{i,<t}) / \pi_{\theta_{\text{old}}}(o_{i,t} \mid \boldsymbol{q}, \boldsymbol{o}_{i,<t})$
GSPO (2025)	Sequence	$s_i(\theta) = \left(\pi_{\theta}(y_i \mid \boldsymbol{x}) / \pi_{\theta_{\text{old}}}(y_i \mid \boldsymbol{x}) \right)^{1/ y_i }$

Controlling drift with a KL term

The Kullback–Leibler (KL) divergence measures how far the current policy has drifted from a fixed **reference policy** — the original model at the start of training. Adding it to the loss acts as a regularizer, discouraging updates that boost reward while also sharply increasing divergence from the reference.

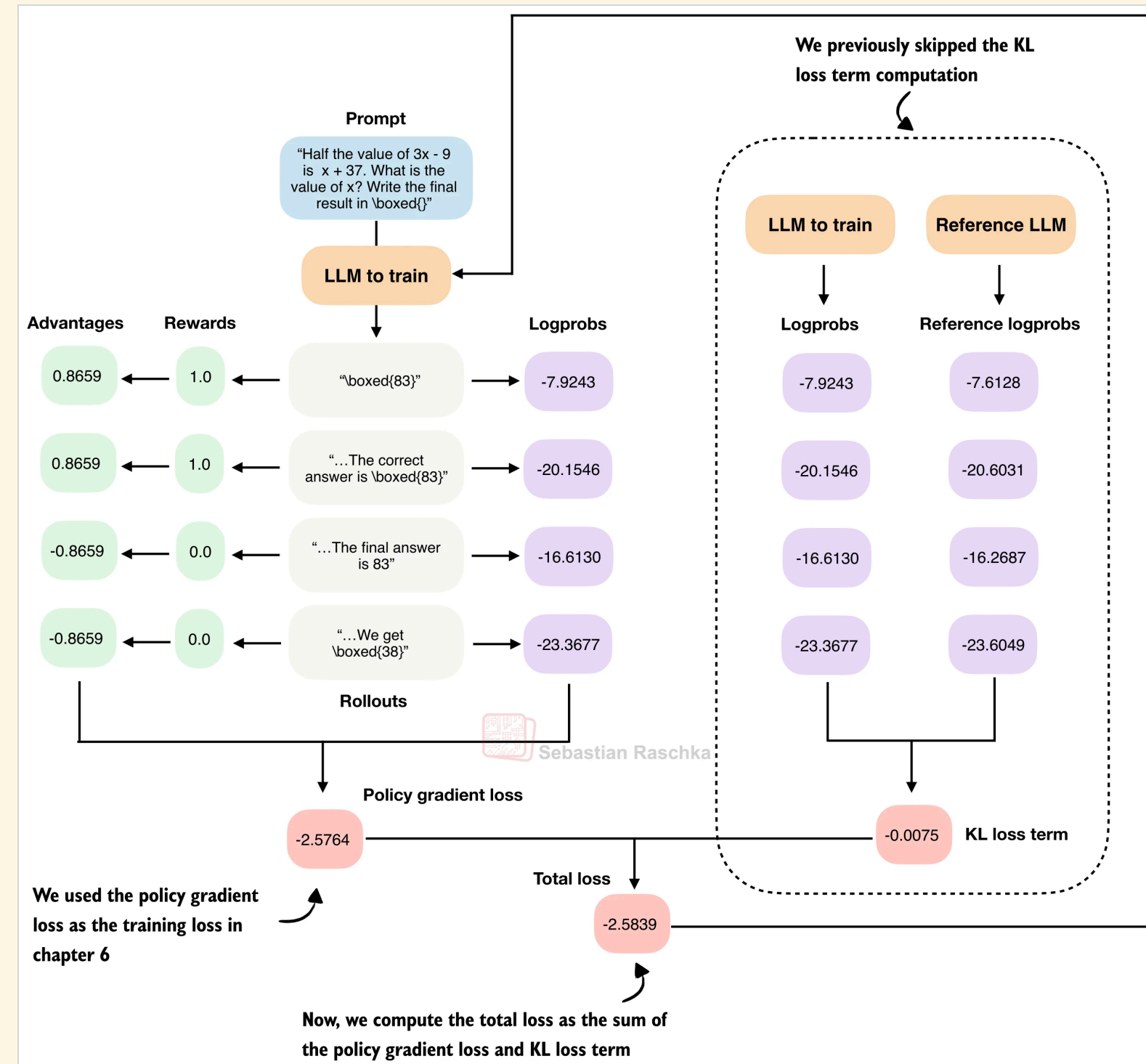
INTUITION: AN ANCHOR, NOT A LEASH

Chasing reward with no constraint can lead a policy to degenerate strategies that score well on a narrow verifier while wrecking its general fluency and coherence — a form of reward hacking. The KL term keeps the policy tethered to a version of itself that could still write ordinary, coherent text, trading some reward-chasing freedom for protection against drifting into gibberish.

RELATED TO, BUT DIFFERENT FROM, CLIPPING

Clipped ratios bound the size of a single update *step*. The KL term instead penalizes long-term *drift* across many steps — the two address complementary risks. In DeepSeek-R1, the reference model itself is periodically refreshed, every 400 steps, to the most recent policy.

Verified: computing the KL loss term



$$\text{kl_loss} = \text{kl_coeff} \times \text{mean}(\text{logprob}_{\text{new}} - \text{logprob}_{\text{ref}})$$
, computed on the exact same generated tokens under both models. The total loss is simply the policy gradient loss plus this KL term: here, $-2.5764 + (-0.0075) = -2.5839$.

Why the KL term can backfire

Adding a naive KL term to real GRPO training caused the run to collapse shortly after step 50 — the model started outputting non-sensical text and random tokens. Two compounding causes:

1. IT SCALES WITH SEQUENCE LENGTH

The KL term is computed on *summed* log-probabilities, so longer responses automatically produce a larger-magnitude KL term — incentivizing ever-longer outputs and destabilizing training.

2. IT CAN BECOME THE ONLY GRADIENT

Once rewards collapse to all-zero, advantages are all zero and the policy gradient loss vanishes entirely — leaving the KL term as the sole remaining signal, which then pushes the model toward near-uniform, effectively random output.

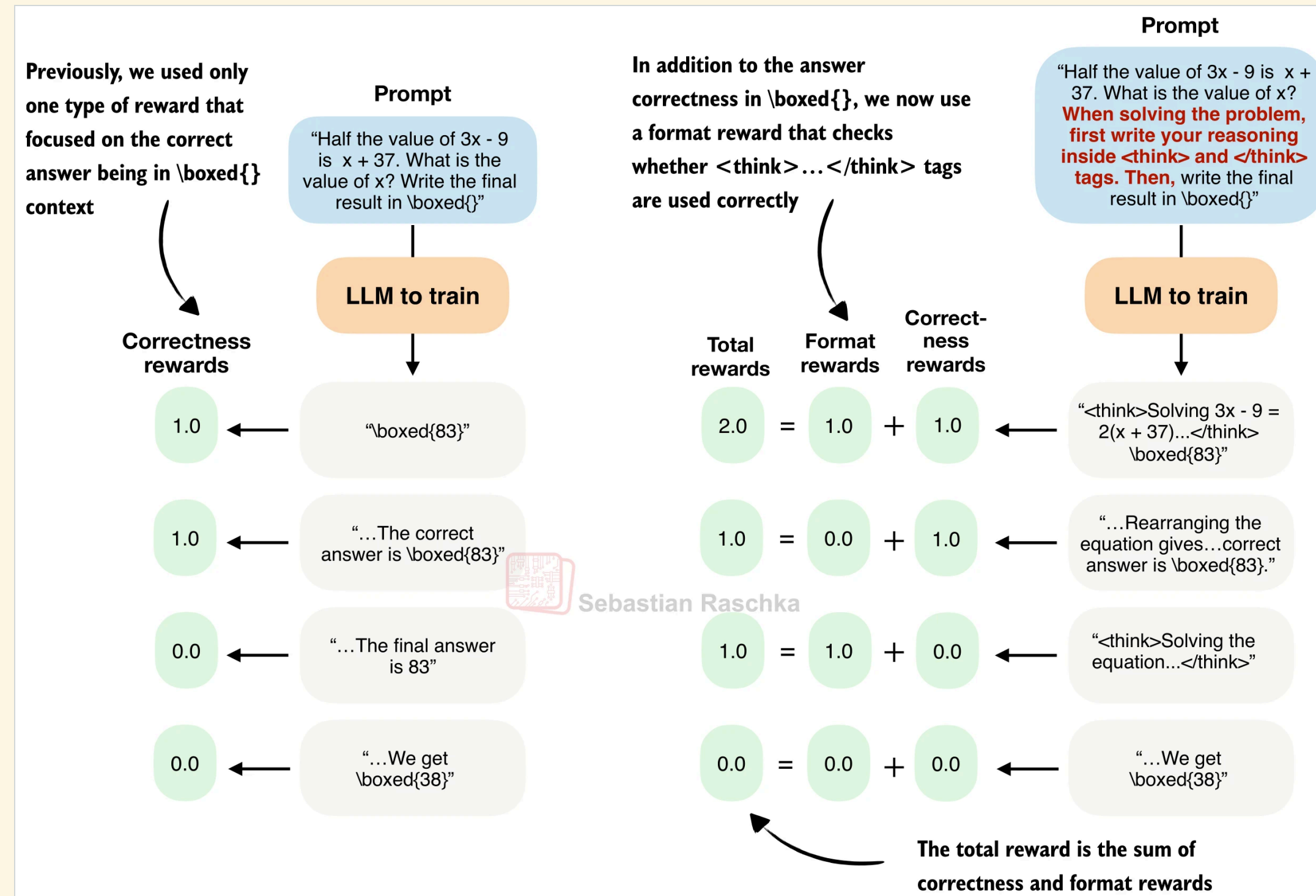
The common recommendation: skip KL for math RL

Several possible fixes exist — length-normalizing the KL and policy-ratio terms, lowering `kl_coeff`, reweighting the KL term via importance sampling, or adding a small format reward so advantages don't collapse to exactly zero.

BUT THE MORE COMMON FIX: DROP IT ENTIRELY

Multiple independent lines of work — **Dr. GRPO**, **Olmo 3**, and **DeepSeek V3.2** — report training better *without* a KL term when training on math data. This also simplifies the implementation and removes the need to keep an extra copy of the reference model in memory during training.

Adding an explicit format reward



DeepSeek-R1 and Qwen3 use `<think></think>` tags to separate intermediate reasoning from the final answer. A simple **format reward** checks whether both tags appear, in the right order, and returns 1.0 if so, else 0.0 — the **total reward** is the sum of the correctness reward and this format reward. Intuitively, this is reward shaping: like giving partial credit for showing work in the right place, it nudges the model toward a useful structure while the correctness reward keeps doing the heavy lifting on actually solving the problem.

A pitfall: over-rewarding format over correctness

In practice, adding a format reward improved training initially, but performance degraded after about 100 steps — correlated with the model producing shorter responses. One explanation: the model was already earning substantial reward just from following the `<think>` format, and stopped focusing enough on actually being correct.

TWO FIXES

Reduce the format reward's weight relative to the correctness reward (e.g. from 1.0 down to 0.1). Or make the format reward **conditional** on correctness — `format_reward *= rlvr_reward` — so following the format only pays off when the final answer is also right.

Further GRPO refinements

In the months after DeepSeek-R1, many further modifications to GRPO were proposed to improve stability and performance:

- **Zero gradient-signal filtering & active sampling & token-level loss & clip-higher** (DAPO)
- **No KL loss** (DAPO; Dr. GRPO) and **no std normalization** of advantages (Dr. GRPO)
- **Truncated importance sampling** for off-policy correction (Yao et al.)
- **Domain-specific KL strengths** (zero for math), **reweighted KL**, and **off-policy sequence masking** (DeepSeek V3.2)
- **Sequence-level importance sampling and clipping** (GSPO); **clipping importance weights rather than token updates** (CISPO)

Reading list

- [Raschka, *Build a Reasoning Model \(From Scratch\)*, Chapter 6](#) — the source for this deck's RL/RLHF/RLVR intro and the full GRPO worked example.
- [Raschka, *Build a Reasoning Model \(From Scratch\)*, Chapter 7](#) — metrics tracking, clipped ratios, KL term, and format rewards.
- [Shao et al., *DeepSeekMath*](#) — introduces GRPO.
- [DeepSeek-AI, *DeepSeek-R1*](#) — RLVR with GRPO at scale.
- [Schulman et al., *Proximal Policy Optimization Algorithms*](#) — source of the clipped-ratio objective.
- [Yu et al., *DAPO*](#) and [Liu et al., *Dr. GRPO*](#) — stability-focused GRPO variants.
- [Zheng et al., *GSPO*](#) and [MiniMax et al., *CISPO*](#) — sequence-level importance sampling variants.
- [Raschka, *The State of Reinforcement Learning for LLM Reasoning*](#) — a detailed PPO vs. GRPO comparison.